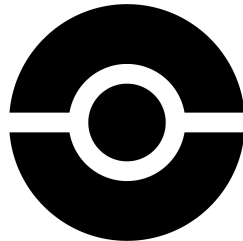


Pokedex App

Zofia Jedlińska

21 January 2026



I. Concept design and requirements

1. Definition of the Project Topic

Project Title: Pokedex Database System

Description:

The project involves the design and implementation of a relational database system combined with a web-based client application dedicated to managing data related to Pokémons. The system serves as a digital encyclopedia (because that's what Pokedex is in the games) that aggregates comprehensive data regarding Pokemon species, their combat statistics, elemental types, move sets, and evolutions.

Goals and Tasks:

The primary goal of the project is to create a data structure that ensures data integrity and retrieval of relationships between Pokémons (e.g., type Effectiveness or evolutionary chains). The main tasks realized by the system include:

- **Data Aggregation:** Storing detailed statistics (HP, Attack, Defense, etc.) and descriptive data for Pokemon across different generations.
- **Relationship Management:** Handling Many-to-Many and One-to-Many relations between Pokemon and their moves, evolution lines, generations, as well as user accounts and their personalized "Pokemon Party".
- **Game Logic Simulation:** Calculating type effectiveness (e.g., Water vs. Fire) and validating game rules (e.g., a Pokemon cannot have two identical types).
- **User Management:** Providing secure access levels for Administrators (content management) and Common Users (team building and browsing).

2. User Requirements Analysis

The system is designed to support three categories of interaction: Guest (not logged in), Common User ("Trainer"), and Administrator.

Functional Requirements:

- **Browsing and Searching:**
 - Users must be able to view a list of all Pokemon and filter them by Name, Elemental Type or Generation.
 - Users must be able to view a list of Moves and filter them by Type, Category, Name or Generation.

- Users must be able to view detailed information about a specific Pokemon, including its base stats, evolutions, moves, details with description and who added the record.
- **Type Effectiveness Calculator:**
 - The system must allow users to select an attacking type and a defending type to calculate the damage multiplier (e.g., 2.0x, 0.5x, 0.0x).
- **User Authentication:**
 - The system must support User Registration and Login using secure password hashing.
 - The system must distinguish between Admin and Common roles.
- **"My Party" Management (for logged-in Users:**
 - Logged-in users can add Pokemon to their personal "Party".
 - The system must enforce a hard limit of maximum 6 Pokemon per user party (same as in the games).
- **Content Management (Admin):**
 - Administrators must have the ability to add new Pokemon and Moves to the database and/or edit the records.

3. Function Design (Database Functions)

The database backend is designed to handle data validity, logic enforcement, and complex reporting through specific database objects.

Basic Functions Realized by the Database:

- **Data Validation:**
 - **Stat Validation:** Constraints (CHECK) ensure that numerical statistics (HP, Attack, Speed, Weight, Height) are non-negative.
 - **Probability Validation:** Accuracy of moves is constrained between 0 and 1.
 - **Enumerated Types:** Use of ENUM for element_type and user_role restricts input to predefined values, preventing typo-based errors.
- **Data Consistency & Logic:**
 - **Type Consistency:** A stored function (check_duplicate_types) and trigger prevent a Pokemon from having the same Primary and Secondary type (e.g., a Pokemon cannot be Fire/Fire).
 - **Business Logic Enforcement:** A stored function (check_party_limit) and trigger (trg_check_party_limit) enforce the rule that a user cannot add more than 6 Pokemon to their party table (user_party).
- **Reporting and Analysis (Views):**
 - **v_type_statistics:** Aggregates data to show the count of Pokemon per type and their average stats, ordered by attack power.
 - **v_generation_counts:** Summarizes how many Pokemon belong to each generation.
 - **v_top_moves:** Identifies the top 10 most common moves learned by Pokemon.

- **v_strong_types:** A complex view using HAVING to filter specific Pokemon types that have an average total statistic greater than 300.
- **Relational Integrity:**
 - Use of Foreign Keys with ON DELETE CASCADE ensures that if a User or Pokemon is deleted, their associated data (like Party entries or Evolution links) are automatically cleaned up to prevent orphaned records.

II. Diagrams project

4. Construction and Analysis of the Data Flow Diagram (DFD)

The system operates as a web-based application where the flow of information is controlled by the user's interaction with the interface (GUI) and processed by the server (PHP) to retrieve or store data in the PostgreSQL database.

- **Guest:** Can search and view Pokemon, Moves, and use the Type Chart.
- **Common User ("Trainer"):** Inherits Guest permissions, plus can log in and manage their "My Party".
- **Administrator:** Inherits User permissions, plus can modify the database (add new Pokemon/Moves).

Data Flow Components:

1. **Inputs:**
 - **Search Query:** Keywords entered by the user to filter Pokemon or Moves.
 - **Authentication Credentials:** Username and Password for registration or login.
 - **Selection:** Choosing specific Pokemon types for the effectiveness calculator or adding a Pokemon to the user's party.
 - **Admin Data:** New Pokemon stats or Move details entered via forms.
2. **Operations:**
 - **Authentication Process:** Verifies hashed passwords against the users table.
 - **Filtering & Retrieval:** SQL queries (SELECT) filter data based on user input.
 - **Validation Logic:** Database triggers ensure data integrity (e.g., preventing >6 Pokemon in a party, checking negative stats).
 - **Calculation:** The application logic computes type effectiveness multipliers based on the type_effectiveness table.
3. **Storage (Data Stores):**
 - **User:** Stores credentials and roles.
 - **Pokedex:** Stores data about Pokemon, Moves, Generations, and Evolutions.
4. **Outputs (Information Display):**
 - **Reports:** Lists of Pokemon, detailed Stat Cards, and Move Lists.
 - **Feedback:** Success/Error messages.
 - **Visuals:** Images of Pokemon and Type icons.

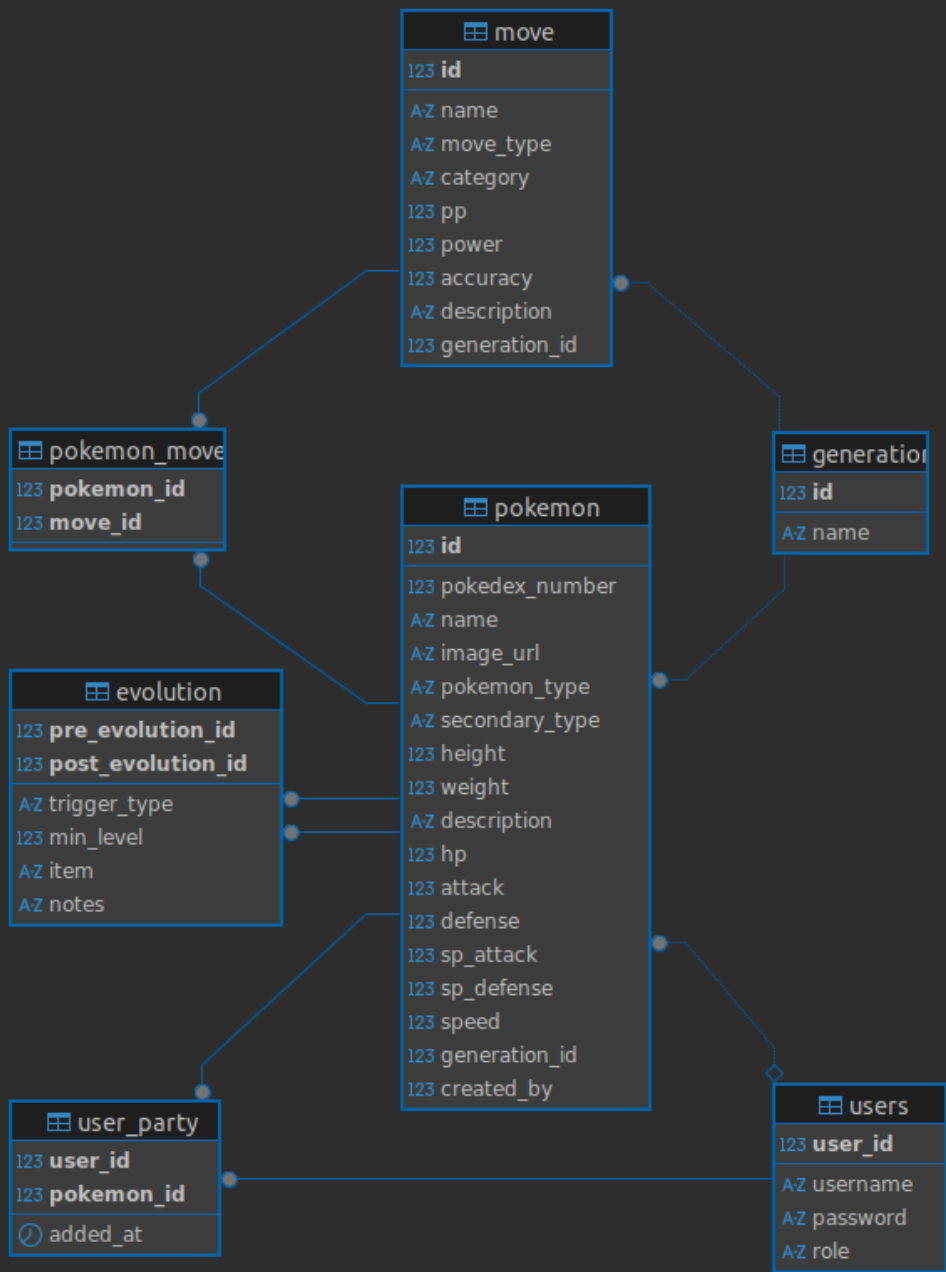
5. Definition of Entities and Attributes

- **users:** Represents the application users.
 - user_id (PK): Unique identifier.
 - username: User's display name.
 - password: Securely hashed password.
 - role: Access level ('Admin' or 'Common').
- **generation (dictionary table):** Informs in which generations the pokemon/move was introduced
 - id (PK): Unique identifier.
 - name: Name of the generation (e.g., "I", "II").
- **pokemon:** Represents the core creature data.
 - id (PK): Unique database identifier.
 - pokedex_number: Official index number.
 - name: Name of the Pokemon.
 - pokemon_type: Primary elemental type.
 - secondary_type: Secondary elemental type (optional).
 - stats: hp, attack, defense, sp_attack, sp_defense, speed.
 - physical_traits: height, weight.
 - generation_id (FK): Link to the generation table.
 - created_by (FK): Link to the user who added the entry (for auditing).
- **move:** Represents abilities used in combat.
 - id (PK): Unique identifier.
 - name: Unique name of the move.
 - move_type: Elemental type of the move.
 - category: Physical, Special, or Status.
 - stats: power, accuracy, pp (Power Points).
 - generation_id (FK): Link to the generation table.
- **evolution:** Represents the evolutionary path between Pokemons.
 - pre_evolution_id (FK/PK Component): The starting Pokemon.
 - post_evolution_id (FK/PK Component): The resulting Pokemon.
 - trigger_type: Mechanism of evolution (Level, Item, etc.).
 - min_level: Level required (optional).
- **type_effectiveness:** Defines the combat logic.
 - attacking_type (PK Component).
 - defending_type (PK Component).
 - multiplier: Damage factor (e.g., 2.0).

6. Relationship Design (ERD Analysis)

Key Relationships:

1. **Generation to Pokemon/Moves (1:N):**
 - One generation can be associated with many pokemon and many moves.
 - Each pokemon/move was introduced in one generation (obviously).
 - *Implementation:* The generation_id Foreign Key is placed in the pokemon and move tables.
2. **User to Pokemon (1:N):**
 - One user (Admin) can create many pokemon.
 - Each pokemon can be created by just one user.
 - *Implementation:* The created_by FK is in the pokemon table.
3. **Pokemon to Moves (M:N):**
 - A Pokemon can learn many moves, and a move can be learned by many Pokemon.
 - The junction table **pokemon_moves** is created.
 - It contains pokemon_id (FK) and move_id (FK). Together they form a composite Primary Key.
4. **User to Pokemon "Party" (M:N):**
 - A User can select multiple Pokemon, and a Pokemon species can be selected by multiple Users.
 - The junction table user_party is created.
 - It contains user_id (FK) and pokemon_id (FK).
 - A trigger limits the number of entries per user to 6.
5. **Pokemon to Pokemon (Evolution) (Recursive M:N):**
 - A Pokemon can evolve into another Pokemon.
 - The evolution table acts as a bridge connecting the pokemon table to itself.
 - pre_evolution_id refers to pokemon(id) and post_evolution_id refers to pokemon(id).



 v_generation_count
A-Z generation_name
123 pokemon_count

 type_effectiveness
A-Z attacking_type
A-Z defending_type
123 multiplier

 v_strong_types
A-Z pokemon_type
123 pokemon_count
123 avg_total_stats

 v_top_moves
A-Z name
A-Z move_type
123 power
123 learned_by_count

 v_type_statistics
A-Z pokemon_type
123 count
123 avg_hp
123 avg_attack
123 avg_speed

III. Logical Design

7. Design of Tables, Keys, and Indexes

SQL Structure:

See the attached *init.sql* file.

Technical Details:

- **Primary Keys (PK):** Every table possesses a Primary Key.
- **Foreign Keys (FK):** Defined with ON DELETE CASCADE or SET NULL.
- **Indexes:**
 - PostgreSQL automatically creates B-Tree indexes for all Primary Keys and UNIQUE constraints.

8. Data Dictionaries

Custom types (ENUMs):

- **public.element_type:** Restricts values to valid Pokemon types: 'Normal', 'Fire', 'Water', 'Grass', 'Electric' etc.
- **public.user_role:** Restricts user privileges to: 'Admin', 'Common'.

Data Dictionary & Constraints:

Table	Column	Data Type	Constraints / Domain	Description
pokemon	height	NUMERIC(5,2)	CHECK (height > 0)	Height in meters.
pokemon	weight	NUMERIC(5,1)	CHECK (weight > 0)	Weight in kilograms.
pokemon	hp, attack, etc.	INT2	CHECK (value >= 0)	Stats
move	accuracy	NUMERIC(3,2)	CHECK (accuracy >= 0 AND <= 1)	Probability (0.0 to 1.0).
move	pp	INT2	CHECK (pp > 0)	Power Points (move usage count).

users	username	TEXT	UNIQUE, NOT NULL	Unique login identifier.
type_effectiveness	multiplier	NUMERIC(3,1)	DEFAULT 1.0	Damage multiplier.

9. Functional Dependency Analysis and Normalization

The database schema has been analyzed to ensure it meets the **Third Normal Form (3NF)** requirements to minimize redundancy and prevent update anomalies.

1NF (First Normal Form):

- *Condition:* All column values are atomic; no repeating groups.
- *Analysis:* The pokemon table does not have columns like move1, move2, move3. instead, moves are separated into the pokemon_moves junction table. Evolution data is also separated.

2NF (Second Normal Form):

- *Condition:* In tables with composite primary keys, all non-key attributes must depend on the *entire* key.
- *Analysis:*
 - In type_effectiveness (PK: attacking_type, defending_type), the multiplier depends on the specific interaction of *both* types, not just one.
 - In evolution (PK: pre_id, post_id), attributes like trigger_type depend on the specific evolution event defined by the pair.
 - Single-key tables (pokemon, move) satisfy this by definition.

3NF (Third Normal Form):

- *Condition:* No transitive dependencies (non-key attributes depend *only* on the primary key).
- *Analysis:*
 - In pokemon table, generation_id refers to the Generation table. The Generation *name* is stored only in the generation table, not repeated in pokemon.
 - In move table, generation_id follows the same logic.
 - There are no calculations stored as columns (e.g., "Total Stats" is not a column, but calculated in a View).

10. Denormalization (Optional)

Denormalization was not applied.

11. Design of Data Operations.

See the attached init.sql file

IV. Functional Design

12. Interfaces for Presentation, Editing, and Data Handling

Forms and Input Structures:

1. Authentication Forms:

○ Login Form:

- *Fields:* Username and Password.
- *Action:* Submits POST request to index.php with action login. Verify hash against database.

○ Registration Form:

- *Fields:* Username and Password.
- *Action:* Submits POST request to index.php with action register. Creates new user record.

2. Filters:

○ Pokemon Filter:

- *Input:* Text search bar (live filtering by Name).
- *Toggles:* Grid of buttons representing Elemental Types (Fire, Water, etc.). Allows selecting up to 2 types for dual-type filtering.

○ Move Filter:

- *Input:* Text search bar.
- *Toggles:* Buttons for filtering by Move Type (e.g., Electric) and Category (Physical/Special/Status).

3. Data Management Forms (Admin Only):

○ Add Pokemon Form:

- *Structure:* form with fields for name, pokedex number, types, stats, image link, generation and description.
- *Validation:* HTML5 required attributes + Database constraints (e.g., HP >= 0).

○ Add Move Form (add_move.php):

- *Structure:* Fields for Name, Type, Category, Power, Accuracy, PP.

13. Data Visualization (Reports)

The system generates reports based on database Views and filtered queries to present data.

Defined Reports:

- *Type Statistics Report:*
 - *Source:* v_type_statistics View.
 - A table showing Average HP, Attack, and Speed for every elemental type. Useful for finding the strongest types.
- *Generation Census:*
 - *Source:* v_generation_counts View.
 - A summary of how many Pokemon were introduced in each generation.
- *Top Moves Report:*
 - *Source:* v_top_moves View.
 - A ranking of the 10 most widely learned moves in the game.

Strong types Report:

- *Source:* v_strong_types View.
- A ranking with top types which have an average stats total of more than 300.

2. Type Effectiveness Calculator:

- *Source:* public.type_effectiveness table.
- Interactive UI where selecting an Attacker and Defender updates a large multiplier display (e.g., "4x Ultra Effective!").

14. Definition of the Application Control Panel

The "Control Panel" is realized as a **Sticky Navigation Menu** (.menu-sticky-wrapper) available at the top of the interface.

Control Panel Elements:

- **Main Navigation:**
 - **[Pokemons]:** Toggles the Pokemon Grid view.
 - **[Moves]:** Toggles the Moves List view.
 - **[Type Chart]:** Opens the interactive calculator.
 - **[Stats]:** Links to the detailed reports.php page.
 - **[My Party]:** (Logged-in users only) Shows the user's selected team.
- **User Controls:**
 - **Status Indicator:** Displays "Trainer: [Username]" tag.
 - **Action Buttons:**
 - *Guest:* [Sign In] / [Sign Up] buttons toggle the respective forms.
 - *User:* [Logout] button terminates the session.

15. Macros (Simplification of Operations)

In the context of this app, "macros" are implemented as JavaScript helper functions and PHP automation that simplify complex tasks for the user.

1. Auto-Search & Filter (JS):

- *Function:* updatePokeSearch(val) and togglePokeType(type).
- Automatically re-renders the DOM grid whenever a key is pressed or a button is clicked, without reloading the page. It abstracts the complex logic of matching dual-types against the dataset.

2. Type Chart Calculation (JS):

- *Function:* calculateChart().
- Automatically computes the product of type multipliers (e.g., Fire attacking Grass/Bug = $2.0 * 2.0 = 4.0x$) instantly upon selection, hiding the underlying math from the user.

3. Session & Role Management (PHP):

- On every page load, the system automatically checks `$_SESSION['role']`. If the user is an 'Admin', CRUD buttons (Add, Edit, Delete) are automatically injected into the HTML. If not, they are removed to prevent unauthorized access attempts.

4. Party Limit Enforcement (DB Trigger):

- The trigger `trg_check_party_limit` acts as a database-level macro. The application simply attempts an INSERT. If the limit (6) is reached, the database automatically rejects it and returns an error, simplifying the application code logic.

VI. Documentation

16. Data Entry

The system supports three primary methods of data entry:

1. Initial Import (Script-based):

- **Method:** execution of SQL INSERT statements via the init.sql script.
- populates static dictionaries (generation), core game data (pokemon, move, type_effectiveness, evolution), and default user accounts (admin/common).
- Performed immediately by the database engine (Constraints, Foreign Keys).

2. Manual Administration:

- **Method:** Web forms (add_pokemon.php, add_move.php) accessible only to users with the 'Admin' role.
- Adding new content to the database without needing SQL knowledge.
- Front-end HTML5 validation + Back-end Database constraints.

3. User Interaction (GUI):

- **Method:** Registration forms and "Add to Party" heart-shaped buttons.
- Creating new user accounts and managing the user_party relationship table.
- PHP-level checks (e.g., duplicate username check) and Database Triggers (e.g., Party limit of 6).

17. User Documentation (User Manual)

1. Getting Started

My application is built using docker. You can open it on any computer as long as Docker is install on it.

1. Install Docker on your computer
2. Open the project folder with an integrated terminal
3. Type "docker compose up --build -d" in the command line and hit enter.
4. Follow this link to open the application in your web browser:
<http://localhost:8080/index.php>

The default view is the Guest Mode, which allows full read-access to the Pokedex data.

2. Features for All Users (Guests)

- **Browsing Pokemon:**
 - Use the "Pokemons" tab to view the grid.
 - Use the Search Bar to find a Pokemon by name.
 - Click the Type Buttons (e.g., Fire, Water) to filter the list. You can select up to two types simultaneously.
 - Click a Pokemon tile to view full details (Stats, Evolutions).
- **Browsing Moves:**
 - Use the "Moves" tab. Filter by Type or Category (Physical/Special) to find specific attacks.
- **Type Chart:**
 - Select an Attacker Type and one or two Defender Types to see the damage multiplier (e.g., "Super Effective!").

3. Features for “Trainers” (Registered Users)

- **Sign Up / Log In:** Use the buttons in the top-right corner to create an account.
- **My Party:**
 - Navigate to a Pokemon's detail page.
 - Click the **"Add to Party"** button (heart-shaped icon).
 - Go to the **"My Party"** tab to view your team.
 - *Note:* You are limited to 6 Pokemon. To add more, you must remove existing ones.

4. Features for Administrators

- **Log in** with an account having the 'Admin' role.
- Green buttons labeled "+ Add Pokemon" and "+ Add Move" will appear at the top of the respective tabs.
- Fill out the forms to permanently add new content to the database.
- “Admin” accounts can be created only directly through the database

18. Technical Documentation

System Architecture

- **Backend:** PHP
- **Database:** PostgreSQL (ver 18.0).
- **Frontend:** HTML5, CSS and JavaScript

Code Structure

- **init.sql**: Contains the schema definitions and initial data. Must be run once to set up the environment.
- **index.php**: The main controller and view. Handles session start, database connection setup, authentication logic, and rendering of the main SPA interface.
- **reports.php**: A separate view dedicated to analytical queries (Views).
- **add_*.php**: Forms for Admin data entry.

Extensibility

The system is designed for modular expansion:

- **Adding Abilities**: To implement Pokemon Abilities (e.g., "Intimidate"), a new table **ability** and a junction table **pokemon_ability** could be created. No changes to existing tables are required
- **Adding Locations**: To implement Locations (e.g., "Cerulean Cave", "Route 5"), a new table **location** and a junction table **pokemon_location** could be created. No changes to existing tables are required
- **Adding translations**: Changes to user interface could be applied so the app could be viewed in different languages. Perhaps AI could be used for dynamic translations (to avoid adding records in 24 languages at once).
- **Items tab**: To add items data a new table **items** could be added. (id, name, description, price). It could be also connected to the **location** table by a junction table **item_location** to deliver information about where to find the item.

19. Bibliography

1. **PostgreSQL Documentation**: *Official Documentation*. <https://www.postgresql.org/docs/>
2. **Bulbapedia**: *Source of Pokémon data (Stats, Moves, Evolutions)*. <https://bulbapedia.bulbagarden.net/>

init.sql file

```

-- types
DO $$ BEGIN
    CREATE TYPE public.element_type AS ENUM (
        'Normal', 'Fire', 'Water', 'Grass', 'Electric', 'Ice',
        'Fighting', 'Poison', 'Ground', 'Flying', 'Psychic',
        'Bug', 'Rock', 'Ghost', 'Dragon', 'Steel', 'Fairy', 'Dark', 'Level', 'Item',
        'Other'
    );
EXCEPTION
    WHEN duplicate_object THEN null;
END $$;

DO $$ BEGIN
    CREATE TYPE public.user_role AS ENUM ('Admin', 'Common');
EXCEPTION WHEN duplicate_object THEN null; END $$;

-- creating tables

-- users (we add the first two users "admin" and "common" in the index.php file
because password hashing wasn't working for me via the database)
-- passwords are hashed
-- two roles: common and admin
CREATE TABLE IF NOT EXISTS public.users (
    user_id SMALLSERIAL PRIMARY KEY,
    username TEXT NOT NULL UNIQUE,
    password TEXT NOT NULL,
    role public.user_role NOT NULL DEFAULT 'Common'
);

-- generations
CREATE TABLE IF NOT EXISTS public.generation (
    id SMALLSERIAL PRIMARY KEY,
    name TEXT NOT NULL
);

-- pokemons
-- data validation, requirement 6b
CREATE TABLE IF NOT EXISTS public.pokemon (
    id SMALLSERIAL PRIMARY KEY,
    pokedex_number INT2 NOT NULL,
    name TEXT NOT NULL,
    image_url TEXT,

```

```

pokemon_type public.element_type NOT NULL,
secondary_type public.element_type,
height NUMERIC(5, 2) NOT NULL CONSTRAINT check_height_pos CHECK (height > 0),
weight NUMERIC(5, 1) NOT NULL CONSTRAINT check_weight_pos CHECK (weight > 0),
description TEXT NOT NULL,
-- validation must be non-negative)
hp INT2 NOT NULL CONSTRAINT check_hp_pos CHECK (hp >= 0),
attack INT2 NOT NULL CONSTRAINT check_atk_pos CHECK (attack >= 0),
defense INT2 NOT NULL CONSTRAINT check_def_pos CHECK (defense >= 0),
sp_attack INT2 NOT NULL CONSTRAINT check_spa_pos CHECK (sp_attack >= 0),
sp_defense INT2 NOT NULL CONSTRAINT check_spd_pos CHECK (sp_defense >= 0),
speed INT2 NOT NULL CONSTRAINT check_spe_pos CHECK (speed >= 0),
generation_id INT2 NOT NULL,
created_by INT4 -- can be null e.g. for pokemons added via database. then "added by
: System" is displayed in the application
);

-- moves - here is also data validation (requirement 6b)
CREATE TABLE IF NOT EXISTS public.move (
    id SMALLSERIAL PRIMARY KEY,
    name TEXT NOT NULL,
    move_type public.element_type NOT NULL,
    category TEXT NOT NULL,
    pp INT2 NOT NULL CONSTRAINT check_pp_pos CHECK (pp > 0),
    power INT2,
    accuracy NUMERIC(3, 2) CONSTRAINT check_acc_range CHECK (accuracy >= 0 AND accuracy
<= 1),
    description TEXT,
    generation_id INT2 NOT NULL
);

-- associative table for pokemons and their moves
CREATE TABLE IF NOT EXISTS public.pokemon_moves (
    pokemon_id INT4 NOT NULL,
    move_id INT4 NOT NULL,
    PRIMARY KEY (pokemon_id, move_id)
);

-- evolutions
CREATE TABLE IF NOT EXISTS public.evolution (
    pre_evolution_id INT4 NOT NULL,
    post_evolution_id INT4 NOT NULL,

```

```

    trigger_type public.element_type NOT NULL,
    min_level INT4,
    item TEXT,
    notes TEXT,
    PRIMARY KEY (pre_evolution_id, post_evolution_id)
);

-- table for type effectiveness
CREATE TABLE IF NOT EXISTS public.type_effectiveness (
    attacking_type public.element_type NOT NULL,
    defending_type public.element_type NOT NULL,
    multiplier NUMERIC(3, 1) NOT NULL DEFAULT 1.0,
    PRIMARY KEY (attacking_type, defending_type)
);

-- table for building My Party
CREATE TABLE IF NOT EXISTS public.user_party (
    user_id INT4 NOT NULL,
    pokemon_id INT4 NOT NULL,
    added_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (user_id, pokemon_id)
);

-- TRIGGERS - requirement 6c - built-in functions and triggers, data consistency
control, blocking entry of incorrect data values
CREATE OR REPLACE FUNCTION check_duplicate_types()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.secondary_type = NEW.pokemon_type THEN
        RAISE EXCEPTION 'Secondary type cannot be the same as Primary type.';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_types
BEFORE INSERT OR UPDATE ON public.pokemon
FOR EACH ROW
EXECUTE FUNCTION check_duplicate_types();

CREATE OR REPLACE FUNCTION check_party_limit()
RETURNS TRIGGER AS $$

```

```

DECLARE
    party_count INTEGER;
BEGIN
    -- Count how many pokemons the user already has in party
    SELECT COUNT(*) INTO party_count
    FROM public.user_party
    WHERE user_id = NEW.user_id;

    -- If they already have 6 or more, block adding another one
    IF party_count >= 6 THEN
        RAISE EXCEPTION 'Party is full! You can only have 6 Pokémon in your party.';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_party_limit
BEFORE INSERT ON public.user_party
FOR EACH ROW
EXECUTE FUNCTION check_party_limit();

-- foreign keys and relations (requirement 2b - use of foreign keys)

ALTER TABLE public.pokemon
    ADD CONSTRAINT fk_pokemon_generation FOREIGN KEY (generation_id) REFERENCES
public.generation(id),
    ADD CONSTRAINT fk_pokemon_creator FOREIGN KEY (created_by) REFERENCES
public.users(user_id) ON DELETE SET NULL;

ALTER TABLE public.move
    ADD CONSTRAINT fk_move_generation FOREIGN KEY (generation_id) REFERENCES
public.generation(id),
    ADD CONSTRAINT move_name_unique UNIQUE (name);

ALTER TABLE public.pokemon_moves
    ADD CONSTRAINT fk_pm_pokemon FOREIGN KEY (pokemon_id) REFERENCES public.pokemon(id)
ON DELETE CASCADE,
    ADD CONSTRAINT fk_pm_move FOREIGN KEY (move_id) REFERENCES public.move(id) ON
DELETE CASCADE;

```

```

ALTER TABLE public.evolution
    ADD CONSTRAINT fk_evo_pre FOREIGN KEY (pre_evolution_id) REFERENCES
public.pokemon(id) ON DELETE CASCADE,
    ADD CONSTRAINT fk_evo_post FOREIGN KEY (post_evolution_id) REFERENCES
public.pokemon(id) ON DELETE CASCADE;

ALTER TABLE public.user_party
    ADD CONSTRAINT fk_party_user FOREIGN KEY (user_id) REFERENCES public.users(user_id)
ON DELETE CASCADE,
    ADD CONSTRAINT fk_party_pokemon FOREIGN KEY (pokemon_id) REFERENCES
public.pokemon(id) ON DELETE CASCADE;

-- VIEWS - 2a
-- AGGREGATE FUNCTIONS - 2c
-- Extension of views - 6a

-- View 1: Type Statistics
CREATE OR REPLACE VIEW public.v_type_statistics AS
WITH all_types AS (
    SELECT pokemon_type as type, hp, attack, speed FROM public.pokemon
    UNION ALL
    SELECT secondary_type as type, hp, attack, speed FROM public.pokemon WHERE
secondary_type IS NOT NULL
)
SELECT
    type as pokemon_type,
    COUNT(*) as count,
    ROUND(AVG(hp), 1) as avg_hp,
    ROUND(AVG(attack), 1) as avg_attack,
    ROUND(AVG(speed), 1) as avg_speed
FROM all_types
GROUP BY type
ORDER BY avg_attack DESC;

-- View 2: Generation Count
CREATE OR REPLACE VIEW public.v_generation_counts AS
SELECT
    g.name as generation_name,
    COUNT(p.id) as pokemon_count
FROM public.generation g

```

```

LEFT JOIN public.pokemon p ON p.generation_id = g.id
GROUP BY g.name
ORDER BY g.name;

-- View 3: Top Moves
CREATE OR REPLACE VIEW public.v_top_moves AS
SELECT
    m.name,
    m.move_type,
    m.power,
    COUNT(pm.pokemon_id) as learned_by_count
FROM public.move m
JOIN public.pokemon_moves pm ON m.id = pm.move_id
GROUP BY m.id, m.name, m.move_type, m.power
ORDER BY learned_by_count DESC
LIMIT 10;

-- View 4: Strong Types (with HAVING clause - Requirement 6a - types for which average
total stats is greater than 300)
CREATE OR REPLACE VIEW public.v_strong_types AS
SELECT
    pokemon_type,
    COUNT(*) as pokemon_count,
    ROUND(AVG(hp + attack + defense + sp_attack + sp_defense + speed), 1) as
avg_total_stats
FROM public.pokemon
GROUP BY pokemon_type
HAVING AVG(hp + attack + defense + sp_attack + sp_defense + speed) > 300
ORDER BY avg_total_stats DESC;

-- INSERTS

-- Generations
INSERT INTO public.generation (name) VALUES
('I'), ('II'), ('III'), ('IV'), ('V'), ('VI'), ('VII'), ('VIII');

-- Attack effectiveness
INSERT INTO public.type_effectiveness (attacking_type, defending_type, multiplier)
VALUES
('Normal', 'Rock', 0.5), ('Normal', 'Ghost', 0.0), ('Normal', 'Steel', 0.5),

```



```
('Fire', 'Fire', 0.5), ('Fire', 'Water', 0.5), ('Fire', 'Grass', 2.0), ('Fire', 'Ice', 2.0), ('Fire', 'Bug', 2.0), ('Fire', 'Rock', 0.5), ('Fire', 'Dragon', 0.5), ('Fire', 'Steel', 2.0),
('Water', 'Fire', 2.0), ('Water', 'Water', 0.5), ('Water', 'Grass', 0.5), ('Water', 'Ground', 2.0), ('Water', 'Rock', 2.0), ('Water', 'Dragon', 0.5),
('Grass', 'Fire', 0.5), ('Grass', 'Water', 2.0), ('Grass', 'Grass', 0.5), ('Grass', 'Poison', 0.5), ('Grass', 'Ground', 2.0), ('Grass', 'Flying', 0.5), ('Grass', 'Bug', 0.5), ('Grass', 'Rock', 2.0), ('Grass', 'Dragon', 0.5), ('Grass', 'Steel', 0.5),
('Electric', 'Water', 2.0), ('Electric', 'Grass', 0.5), ('Electric', 'Electric', 0.5), ('Electric', 'Ground', 0.0), ('Electric', 'Flying', 2.0), ('Electric', 'Dragon', 0.5),
('Ice', 'Fire', 0.5), ('Ice', 'Water', 0.5), ('Ice', 'Grass', 2.0), ('Ice', 'Ice', 0.5), ('Ice', 'Ground', 2.0), ('Ice', 'Flying', 2.0), ('Ice', 'Dragon', 2.0), ('Ice', 'Steel', 0.5),
('Fighting', 'Normal', 2.0), ('Fighting', 'Ice', 2.0), ('Fighting', 'Poison', 0.5), ('Fighting', 'Flying', 0.5), ('Fighting', 'Psychic', 0.5), ('Fighting', 'Bug', 0.5), ('Fighting', 'Rock', 2.0), ('Fighting', 'Ghost', 0.0), ('Fighting', 'Dark', 2.0), ('Fighting', 'Steel', 2.0), ('Fighting', 'Fairy', 0.5),
('Poison', 'Grass', 2.0), ('Poison', 'Poison', 0.5), ('Poison', 'Ground', 0.5), ('Poison', 'Rock', 0.5), ('Poison', 'Ghost', 0.5), ('Poison', 'Steel', 0.0), ('Poison', 'Fairy', 2.0),
('Ground', 'Fire', 2.0), ('Ground', 'Grass', 0.5), ('Ground', 'Electric', 2.0), ('Ground', 'Poison', 2.0), ('Ground', 'Flying', 0.0), ('Ground', 'Bug', 0.5), ('Ground', 'Rock', 2.0), ('Ground', 'Steel', 2.0),
('Flying', 'Grass', 2.0), ('Flying', 'Electric', 0.5), ('Flying', 'Fighting', 2.0), ('Flying', 'Bug', 2.0), ('Flying', 'Rock', 0.5), ('Flying', 'Steel', 0.5),
('Psychic', 'Fighting', 2.0), ('Psychic', 'Poison', 2.0), ('Psychic', 'Psychic', 0.5), ('Psychic', 'Dark', 0.0), ('Psychic', 'Steel', 0.5),
('Bug', 'Fire', 0.5), ('Bug', 'Grass', 2.0), ('Bug', 'Fighting', 0.5), ('Bug', 'Poison', 0.5), ('Bug', 'Flying', 0.5), ('Bug', 'Psychic', 2.0), ('Bug', 'Ghost', 0.5), ('Bug', 'Dark', 2.0), ('Bug', 'Steel', 0.5), ('Bug', 'Fairy', 0.5),
('Rock', 'Fire', 2.0), ('Rock', 'Ice', 2.0), ('Rock', 'Fighting', 0.5), ('Rock', 'Ground', 0.5), ('Rock', 'Flying', 2.0), ('Rock', 'Bug', 2.0), ('Rock', 'Steel', 0.5),
('Ghost', 'Normal', 0.0), ('Ghost', 'Psychic', 2.0), ('Ghost', 'Ghost', 2.0), ('Ghost', 'Dark', 0.5),
('Dragon', 'Dragon', 2.0), ('Dragon', 'Steel', 0.5), ('Dragon', 'Fairy', 0.0), ('Dark', 'Fighting', 0.5), ('Dark', 'Psychic', 2.0), ('Dark', 'Ghost', 2.0), ('Dark', 'Dark', 0.5), ('Dark', 'Fairy', 0.5),
('Steel', 'Fire', 0.5), ('Steel', 'Water', 0.5), ('Steel', 'Electric', 0.5), ('Steel', 'Ice', 2.0), ('Steel', 'Rock', 2.0), ('Steel', 'Steel', 0.5), ('Steel', 'Fairy', 2.0), ('Fairy', 'Fire', 0.5), ('Fairy', 'Fighting', 2.0), ('Fairy', 'Poison', 0.5), ('Fairy', 'Dragon', 2.0), ('Fairy', 'Dark', 2.0), ('Fairy', 'Steel', 0.5)
```

```

ON CONFLICT (attacking_type, defending_type) DO UPDATE SET multiplier =
EXCLUDED.multiplier;

-- Pokemons - created_by is null, for pokemons added from the database
INSERT INTO public.pokemon (
    pokdex_number, name, image_url, pokemon_type, secondary_type, height, weight,
    description, hp, attack, defense, sp_attack, sp_defense, speed, generation_id
) VALUES
(1, 'Bulbasaur',
'https://archives.bulbagarden.net/media/upload/thumb/f/fb/0001Bulbasaur.png/500px-0001
Bulbasaur.png', 'Grass', 'Poison', 0.7, 6.9, 'A strange seed was planted on its back
at birth.', 45, 49, 49, 65, 65, 45, 1),
(2, 'Ivysaur',
'https://archives.bulbagarden.net/media/upload/thumb/8/81/0002Ivysaur.png/500px-0002Iv
ysaur.png', 'Grass', 'Poison', 1.0, 13.0, 'When the bulb on its back grows large, it
appears to lose the ability to stand on its hind legs.', 60, 62, 63, 80, 80, 60, 1),
(3, 'Venusaur',
'https://archives.bulbagarden.net/media/upload/thumb/6/6b/0003Venusaur.png/500px-0003V
enusaur.png', 'Grass', 'Poison', 2.0, 100.0, 'Its plant blooms when it is absorbing
solar energy. It stays on the move to seek sunlight.', 80, 82, 83, 100, 100, 80, 1),
(4, 'Charmander',
'https://archives.bulbagarden.net/media/upload/thumb/2/27/0004Charmander.png/500px-000
4Charmander.png', 'Fire', NULL, 0.6, 8.5, 'Obviously prefers hot places. When it
rains, steam is said to spout from the tip of its tail.', 39, 52, 43, 60, 50, 65, 1),
(5, 'Charmeleon',
'https://archives.bulbagarden.net/media/upload/thumb/0/05/0005Charmeleon.png/500px-000
5Charmeleon.png', 'Fire', NULL, 1.1, 19.0, 'When it swings its burning tail, it
elevates the temperature to unbearably high levels.', 58, 64, 58, 80, 65, 80, 1),
(6, 'Charizard',
'https://archives.bulbagarden.net/media/upload/thumb/3/38/0006Charizard.png/500px-0006
Charizard.png', 'Fire', 'Flying', 1.7, 90.5, 'Spits fire that is hot enough to melt
boulders. Known to cause forest fires unintentionally.', 78, 84, 78, 109, 85, 100, 1),
(7, 'Squirtle',
'https://archives.bulbagarden.net/media/upload/thumb/5/54/0007Squirtle.png/500px-0007S
quirtle.png', 'Water', NULL, 0.5, 9.0, 'After birth, its back swells and hardens into
a shell. Powerfully sprays foam from its mouth.', 44, 48, 65, 50, 64, 43, 1),
(8, 'Wartortle',
'https://archives.bulbagarden.net/media/upload/thumb/0/0f/0008Wartortle.png/500px-0008
Wartortle.png', 'Water', NULL, 1.0, 22.5, 'Often hides in water to stalk unwary prey.
For swimming fast, it moves its ears to maintain balance.', 59, 63, 80, 65, 80, 58,
1),

```

```

(9, 'Blastoise',
'https://archives.bulbagarden.net/media/upload/thumb/0/0a/0009Blastoise.png/500px-0009
Blastoise.png', 'Water', NULL, 1.6, 85.5, 'A brutal Pokémon with pressurized water
jets on its shell. They are used for high speed tackles.', 79, 83, 100, 85, 105, 78,
1);

-- Moves
INSERT INTO public.move (name, move_type, category, pp, power, accuracy, description,
generation_id) VALUES
('Pound', 'Normal', 'Physical', 35, 40, 1.00, 'Pounds with forelegs or tail.', 1),
('Karate Chop', 'Fighting', 'Physical', 25, 50, 1.00, 'High critical hit ratio.', 1),
('Double Slap', 'Normal', 'Physical', 10, 15, 0.85, 'Repeatedly slaps 2-5 times.', 1),
('Comet Punch', 'Normal', 'Physical', 15, 18, 0.85, 'Repeatedly punches 2-5 times.',
1),
('Mega Punch', 'Normal', 'Physical', 20, 80, 0.85, 'A powerful punch thrown very
hard.', 1),
('Pay Day', 'Normal', 'Physical', 20, 40, 1.00, 'Throws coins. Money is recovered
after battle.', 1),
('Fire Punch', 'Fire', 'Physical', 15, 75, 1.00, 'May burn the opponent.', 1),
('Ice Punch', 'Ice', 'Physical', 15, 75, 1.00, 'May freeze the opponent.', 1),
('Thunder Punch', 'Electric', 'Physical', 15, 75, 1.00, 'May paralyze the opponent.',
1),
('Scratch', 'Normal', 'Physical', 35, 40, 1.00, 'Scratches with sharp claws.', 1),
('Vise Grip', 'Normal', 'Physical', 30, 55, 1.00, 'Grips with powerful pincers.', 1),
('Guillotine', 'Normal', 'Physical', 5, NULL, 0.30, 'A one-hit KO move.', 1),
('Razor Wind', 'Normal', 'Special', 10, 80, 1.00, 'Charges on first turn, attacks on
second.', 1),
('Swords Dance', 'Normal', 'Status', 20, NULL, NULL, 'Sharply raises user Attack.',
1),
('Cut', 'Normal', 'Physical', 30, 50, 0.95, 'Cuts using claws, scythes, etc.', 1),
('Gust', 'Flying', 'Special', 35, 40, 1.00, 'Strikes the target with wings.', 1),
('Wing Attack', 'Flying', 'Physical', 35, 60, 1.00, 'Strikes the target with wings.',
1),
('Whirlwind', 'Normal', 'Status', 20, NULL, NULL, 'Blows away the opponent.', 1),
('Fly', 'Flying', 'Physical', 15, 90, 0.95, 'Flies up on first turn, attacks on
second.', 1),
('Bind', 'Normal', 'Physical', 20, 15, 0.85, 'Binds the target for 2-5 turns.', 1),
('Slam', 'Normal', 'Physical', 20, 80, 0.75, 'Slams the target with a long tail, vine
etc.', 1),
('Stomp', 'Normal', 'Physical', 20, 65, 1.00, 'Stomps on the enemy.', 1),
('Double Kick', 'Fighting', 'Physical', 30, 30, 1.00, 'Kicks twice in a row.', 1),
('Mega Kick', 'Normal', 'Physical', 5, 120, 0.75, 'A powerful kicking attack.', 1),

```

```

('Tackle', 'Normal', 'Physical', 35, 40, 1.00, 'Charges the foe with a full-body
tackle.', 1),
('Growl', 'Normal', 'Status', 40, NULL, 1.00, 'Lowers the foe Attack.', 1),
('Vine Whip', 'Grass', 'Physical', 25, 45, 1.00, 'Strikes the foe with slender
whips.', 1),
('Growth', 'Normal', 'Status', 20, NULL, NULL, 'Raises user Attack and Sp. Atk.', 1),
('Leech Seed', 'Grass', 'Status', 10, NULL, 0.90, 'Steals HP from the foe on every
turn.', 1),
('Razor Leaf', 'Grass', 'Physical', 25, 55, 0.95, 'High critical hit ratio.', 1),
('Poison Powder', 'Poison', 'Status', 35, NULL, 0.75, 'Poisons the foe.', 1),
('Sleep Powder', 'Grass', 'Status', 15, NULL, 0.75, 'Puts the foe to sleep.', 1),
('Seed Bomb', 'Grass', 'Physical', 15, 80, 1.00, 'Slams a barrage of hard seeds at the
target.', 1),
('Take Down', 'Normal', 'Physical', 20, 90, 0.85, 'User receives recoil damage.', 1),
('Sweet Scent', 'Normal', 'Status', 20, NULL, 1.00, 'Lowers the foe Evasion.', 1),
('Synthesis', 'Grass', 'Status', 5, NULL, NULL, 'Restores HP based on weather.', 1),
('Worry Seed', 'Grass', 'Status', 10, NULL, 1.00, 'Changes the foe ability to
Insomnia.', 1),
('Power Whip', 'Grass', 'Physical', 10, 120, 0.85, 'Violently lashes the foe with
vines.', 1),
('Solar Beam', 'Grass', 'Special', 10, 120, 1.00, 'Charges on first turn, attacks on
second.', 1)
ON CONFLICT (name) DO NOTHING;

-- Moves to Pokemon assignments (Bulbasaur)
INSERT INTO public.pokemon_moves (pokemon_id, move_id)
SELECT p.id, m.id
FROM public.pokemon p
CROSS JOIN public.move m
WHERE p.name = 'Bulbasaur'
AND m.name IN (
    'Tackle', 'Growl', 'Vine Whip', 'Growth', 'Leech Seed', 'Razor Leaf',
    'Poison Powder', 'Sleep Powder', 'Seed Bomb', 'Take Down', 'Sweet Scent',
    'Synthesis', 'Worry Seed', 'Power Whip', 'Solar Beam'
)
ON CONFLICT (pokemon_id, move_id) DO NOTHING;

-- Evolutions
INSERT INTO public.evolution (pre_evolution_id, post_evolution_id, trigger_type,
min_level, item, notes) VALUES
(1, 2, 'Level', 16, NULL, NULL),
(2, 3, 'Level', 32, NULL, NULL),

```

```
(4, 5, 'Level', 16, NULL, NULL),  
(5, 6, 'Level', 36, NULL, NULL),  
(7, 8, 'Level', 16, NULL, NULL),  
(8, 9, 'Level', 36, NULL, NULL);
```